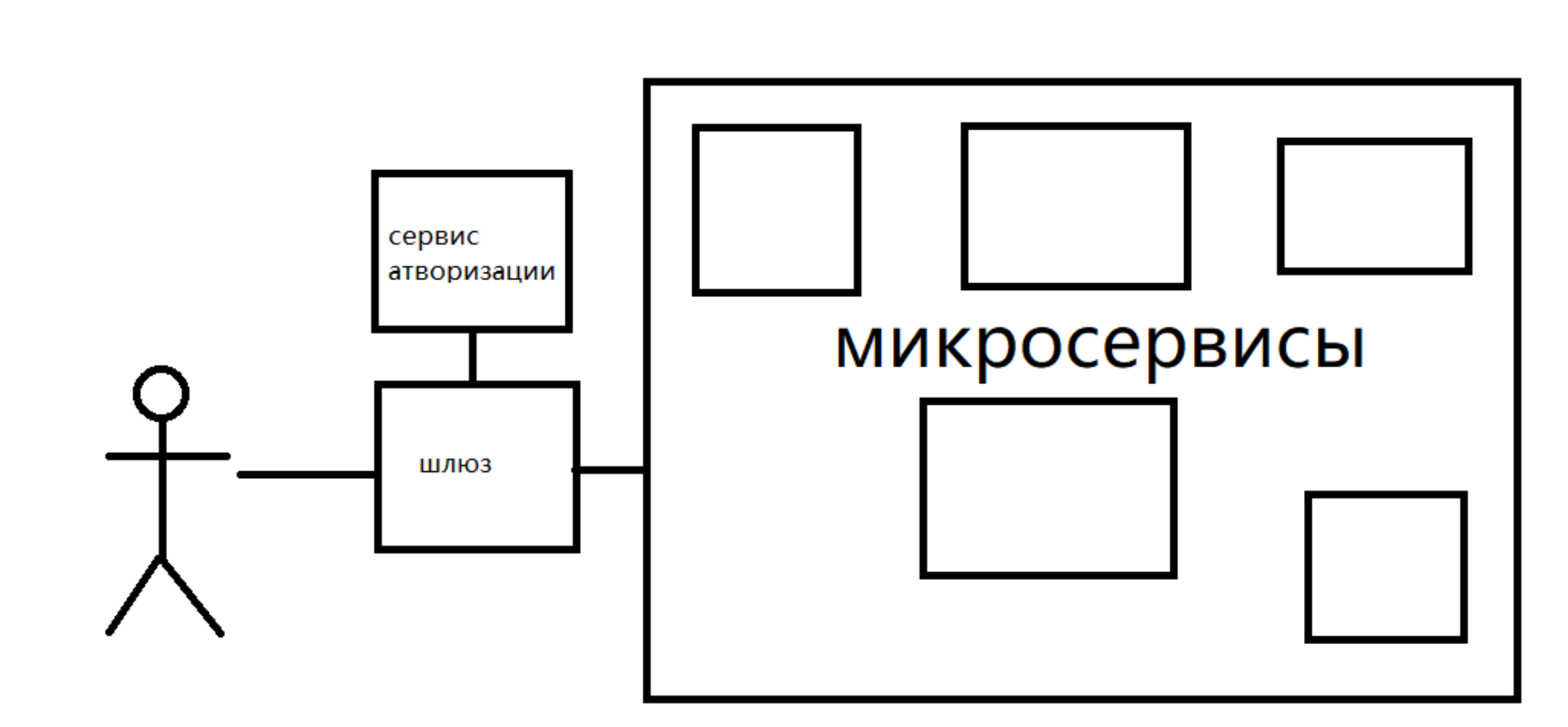
ЗАЩИТА ПРИЛОЖЕНИЙ

Spring security

Микросервисы



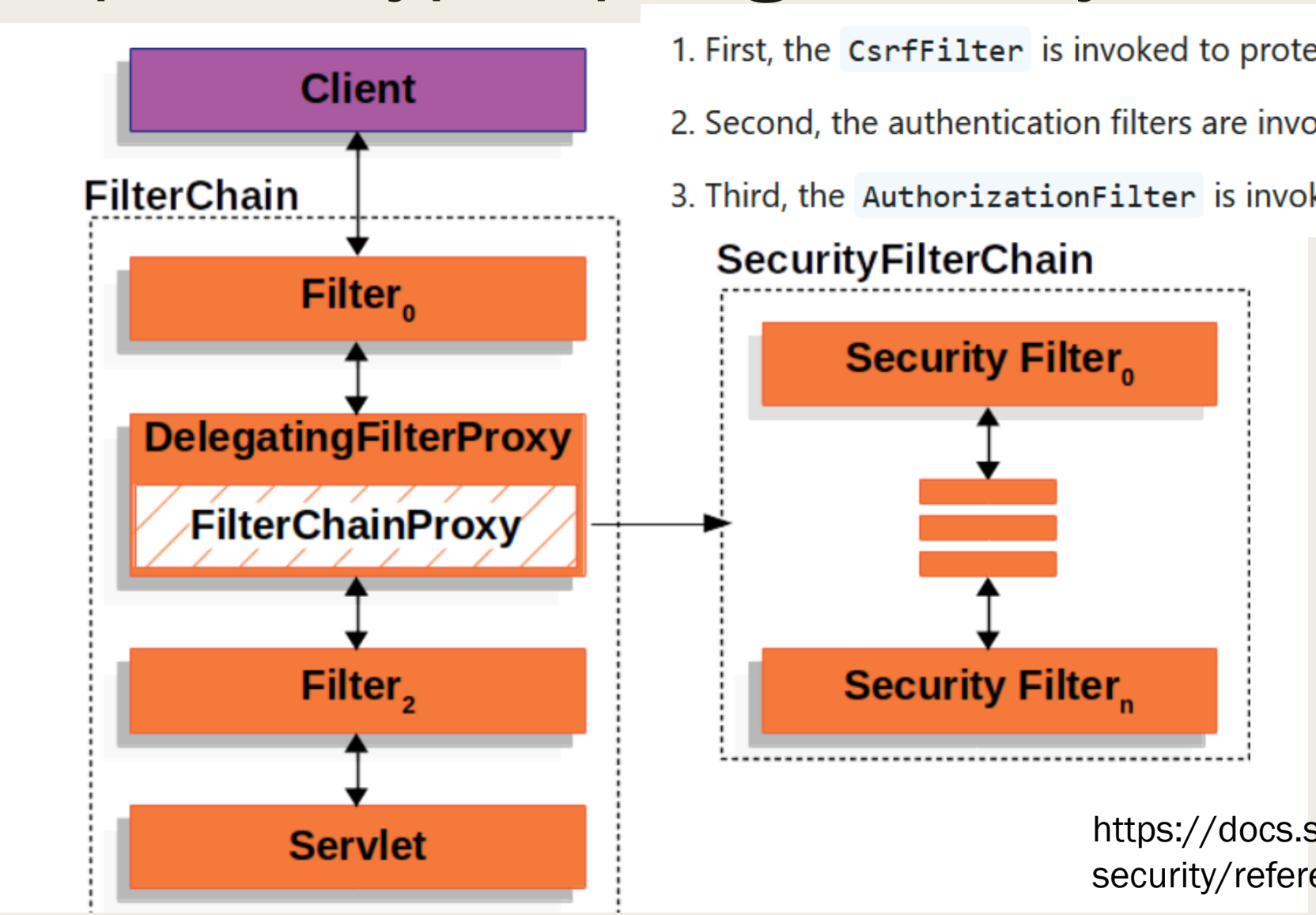
JWT-токен

- https://jwt.io/
- JWT состоит из трех частей: заголовка (header), полезной нагрузки (payload) и подписи (signature). JWT имеет фиксированную структуру, которая включает три части, разделенные точками. Это делает его легко проверяемым и не позволяет добавлять произвольные данные без изменения подписи. Подпись создаётся с использованием алгоритма подписи и секретного ключа (или закрытого ключа в случае асимметричного шифрования).
  - *Секретный ключ: Если используется симметричный алгоритм (например, HMAC), то как отправитель, так и получатель знают этот секретный ключ. Это означает, что только они могут создать и проверить подпись.*
  - *Закрытый и открытый ключи: Если используется асимметричный алгоритм (например, RSA), то токен подписывается закрытым ключом, а проверяется открытым ключом. Это обеспечивает дополнительный уровень безопасности, поскольку только обладатель закрытого ключа может создавать валидные токены.*

JWT-токен

- Если кто-то попытается изменить данные в полезной нагрузке (например, добавить или изменить пользовательские данные), это приведет к изменению подписи. При проверке подписи она не совпадёт, и токен будет признан недействительным.
- JWT поддерживает различные алгоритмы подписи, такие как HMAC, RSA и ECDSA. Эти алгоритмы имеют свои криптографические свойства, которые делают их стойкими к подделке.
- JWT часто содержит информацию о времени жизни (например, `exp` — время истечения). Это ограничивает срок действия токена, что уменьшает риск использования украденного токена. Даже если токен будет скомпрометирован, злоумышленник сможет использовать его только в течение ограниченного времени.
- Хотя сам токен защищён, важно также использовать дополнительные меры безопасности, такие как:
  - *HTTPS для защиты токенов во время передачи*
  - *Проверка на стороне сервера при каждом запросе для подтверждения действительности токена.*
  - *Регулярная ротация ключей и токенов.*

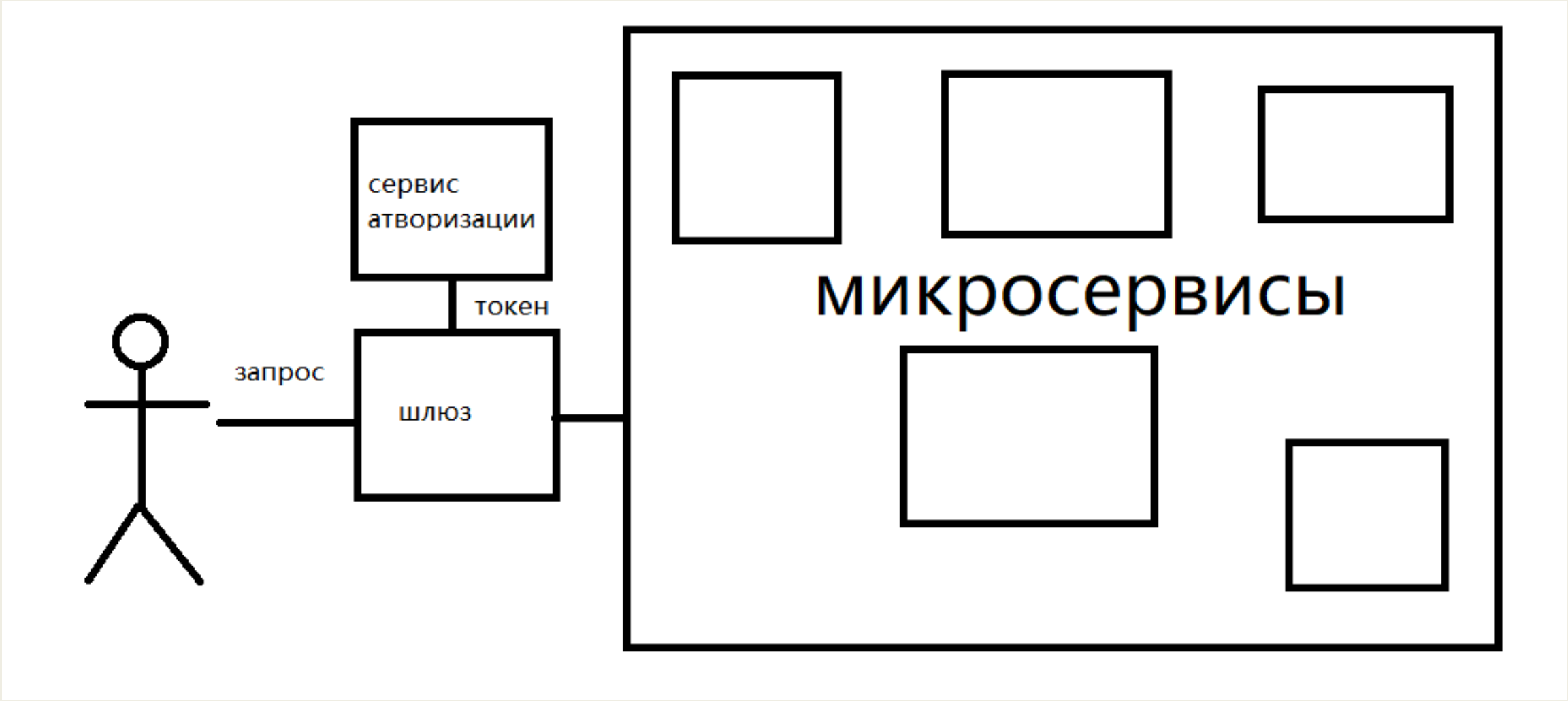
Архитектура spring security



1. First, the `CsrfFilter` is invoked to protect against CSRF attacks.
2. Second, the authentication filters are invoked to authenticate the request.
3. Third, the `AuthorizationFilter` is invoked to authorize the request.

https://docs.spring.io/spring-security/reference/servlet/architecture.html

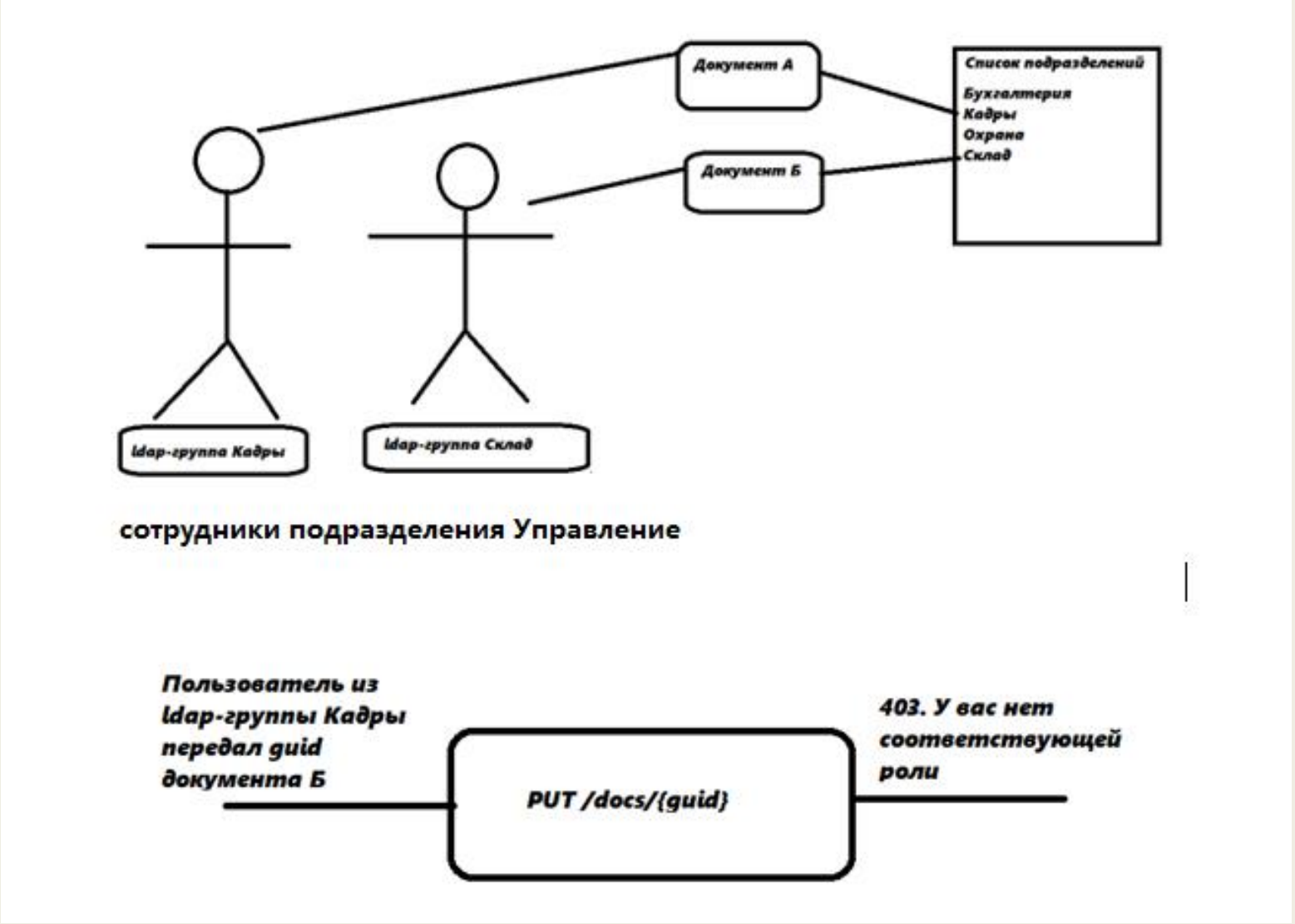
Реализация



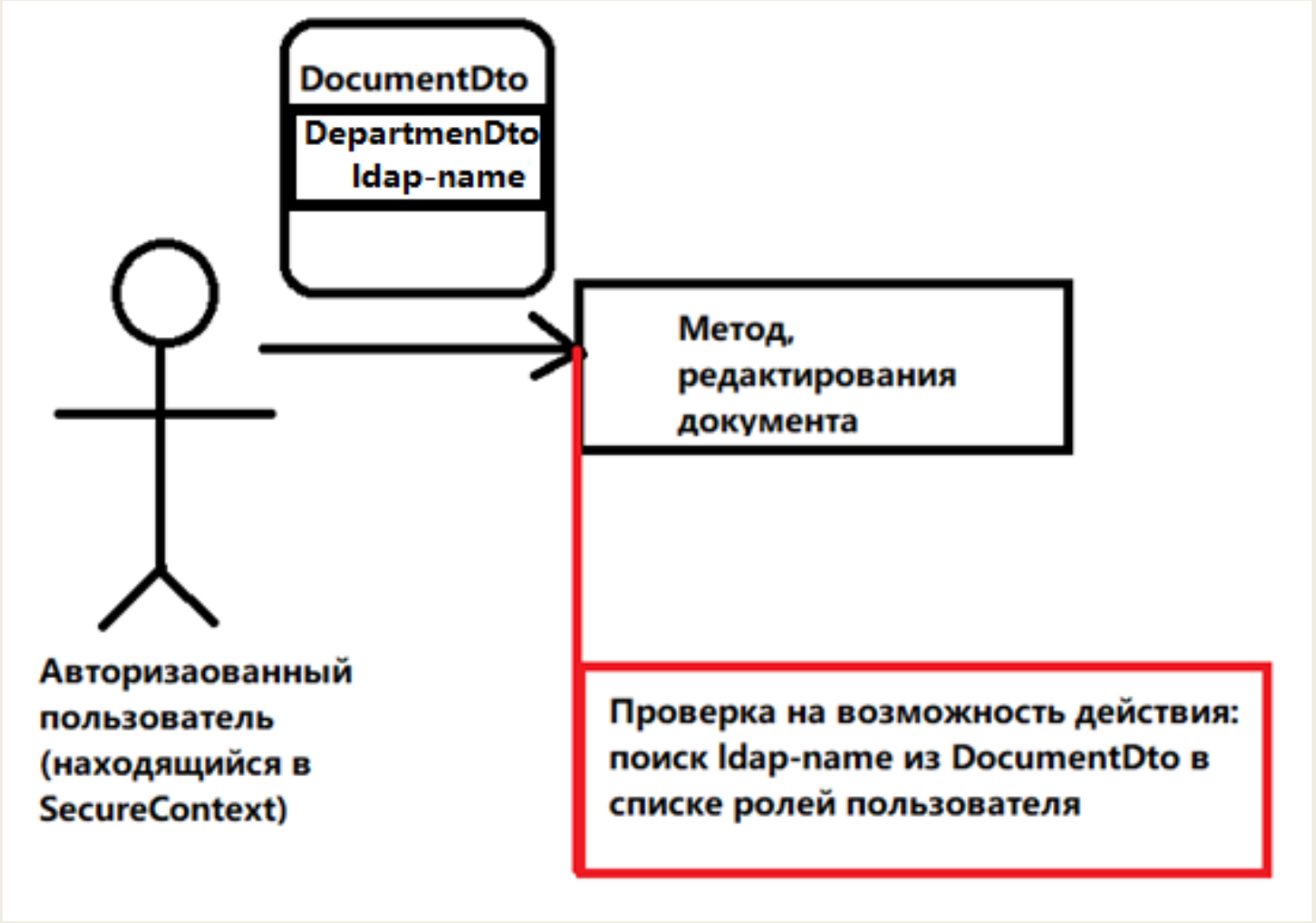
Пример приложения с jwt-токеном

```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt</artifactId>
<version>0.9.1</version> </dependency>
 <dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Более сложный вариант



Что мы хотим?


DocumentDto
DepartmenDto
ldap-name
Метод, редактирования документа
Авторизаованный пользователь (находящийся в SecureContext)
Проверка на возможность действия: поиск ldap-name из DocumentDto в списке ролей пользователя

Как это сделать на аспектах?

```
@Service

public class AuthUserServiceImpl implements IAuthService {

@Override
public Set<String> getMySecureRoles(){

Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
AuthUserDetails authDitails = (AuthUserDetails)authentication.getPrincipal();


return  authDitails.getAuthorities().stream().map(e->e.toString()).collect(Collectors.toSet());     }}
```

```
[
  "MONEY",
  "SKLAD",
  "CONTROL"
]
```

Как это сделать на аспектах?

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface SecuredAction { }
```

Как это сделать на аспектах?

```
@Component
@RequiredArgsConstructor
public class ActionCheckPermissionImpl implements
IActionCheckPermission {

private final IAuthService authService;

private final DocRepo docRepo;

private final DocMapper docMapper;

@Override
public void checkPermission(Object object) {

String ldapName="";

if (object instanceof DocumentDto doc) {

        ldapName=doc.getDepartment().getLdapName();
}
if (object instanceof UUID uuid) {
   Document docForDelete =
docRepo.findById(uuid).orElseThrow(() -> new
NotFoundException("Document with guid %s not
found".formatted(uuid)));

ldapName =
docMapper.toDocDto(docForDelete).getDepartment().getLda
                                pName();
}

if (!authService.getMySecureRoles().contains(ldapName)
   throw new AccessDeniedException("Access denied: you
have not necessary role");

}
```

Как это сделать на аспектах?

```
@Aspect
@Component
@RequiredArgsConstructor
public class ActionCheckPermissionAspect {
    private final IActionCheckPermission checker;

    @Before(value =
"@annotation(ru.mycompany.myproduct.roles.annotation.SecuredAction)")
    public void checkPermissionAction(JoinPoint joinPoint) {

checker.checkPermission(Arrays.stream(joinPoint.getArgs()).toList().get(0));
    }

}
```

Как это сделать на аспектах?

```
public interface DocumentService {

    List<DocumentDto> getDocuments();


@SecuredAction
    DocumentDto addDocument(DocumentDto dto);

    @SecuredAction
    List<DocumentDto> updateDocument(DocumentDto dto);

@SecuredAction
    void deleteDocument(UUID id);

}
```

Как это сделать на аспектах?

То есть, dto, пришедшее в метод может быть таким:

{ department : { ldap-name: “SKLAD”}}

Если getMySecureRoles() вернет пользователю вот такой список: ["MONEY", "SKLAD", "CONTROL"], проверка будет успешно пройдена. Если же вот такой - ["MONEY", "CONTROL"] возникнет AccessDeniedException.

Минусы решения

- Во-первых, в методе checkPermission необходимо прописывать блок приведения к конкретному типу для каждого из проверяемых объектов.
- Во-вторых, текущий вариант не предполагает расширения для контроля методов удаления разных объектов. Потому что по UUID мы не можем понять, какой объект к нам пришел.
- В-третьих, аспектные аннотации в сервисном слое могут не отрабатывать. А для написания тестов это важно.
- В-четвертых, этот механизм не связан с механизмом контроля spring security и инциденты безопасности не попадут в его лог.

Как проверку привязать стандартным механизмом Spring Security? - через @PreAuthorize.

Во-первых, не забыть поставить аннотацию @EnableWebSecurity над основным приложением.

Во-вторых, явно указать имя бина, который осуществляет проверку. В нашем случае это выполняется так:

```
@Component(“checkerBean”)
public class ActionCheckPermissionImpl  implements
IActionCheckPermission

{…}
```

Как проверку привязать стандартным механизмом Spring Security? - через @PreAuthorize.

В-третьих, изменить проверочный метод – он должен возвращать true, если доступ разрешен, и false – в противном случае. Т.е. выбрасывания пользовательского исключения тут не предполагается. Если в доступе отказано, будет выброшено стандартное AccessDenied (без специфической информации) самим spring security. Поэтому если вам крайне важно выбрасывать нестандартное исключение, то вариант с @PreAuthorize не подойдет.

Как проверку привязать стандартным механизмом Spring Security? - через @PreAuthorize.

```
public interface DocumentService {

    List<DocumentDto> getDocuments();

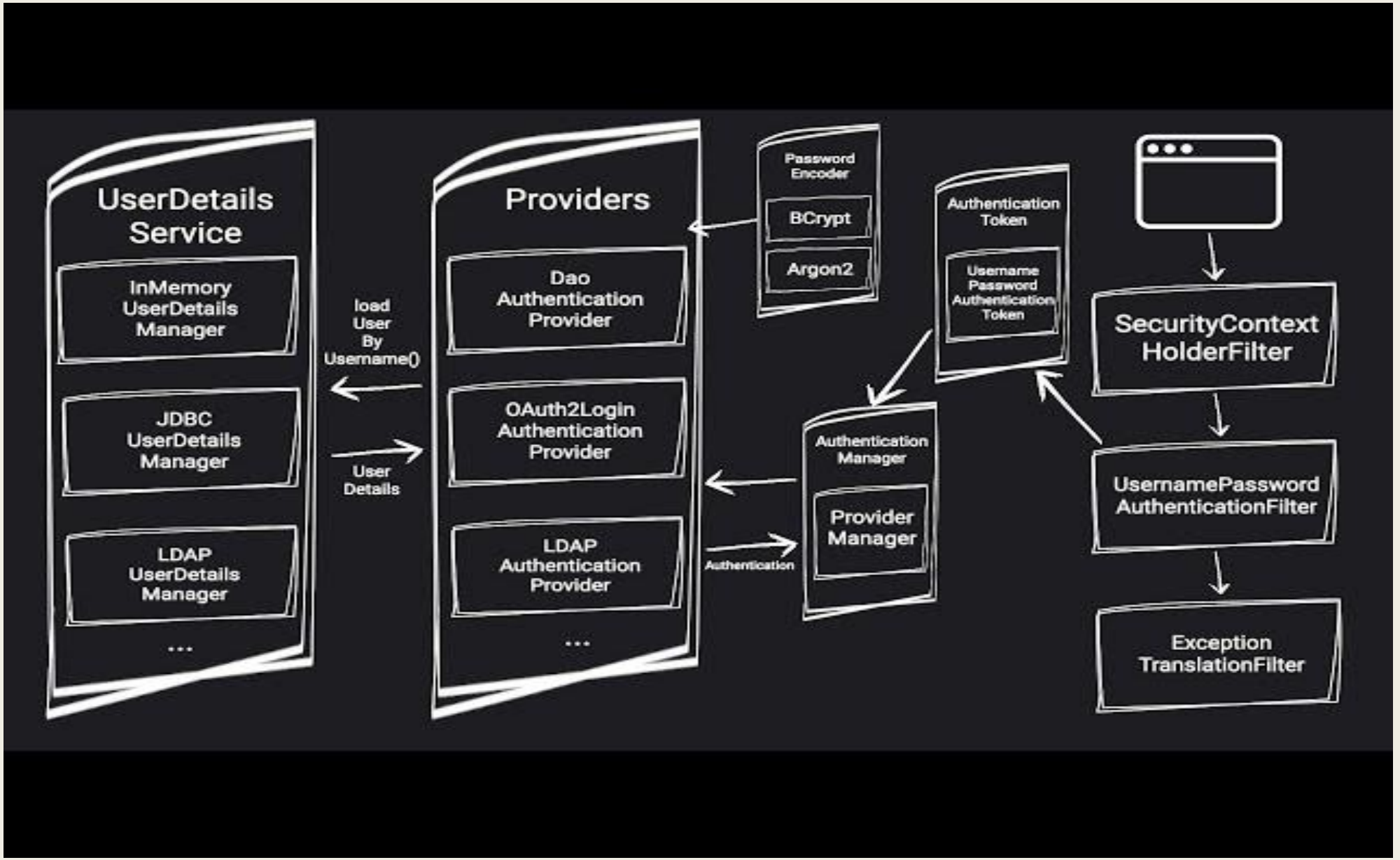

    @PreAuthorize(“@checkerBean. checkPermission(#dto)”)
    DocumentDto addDocument(DocumentDto dto);

    @PreAuthorize(“@checkerBean. checkPermission(#dto)”)
    List<DocumentDto> updateDocument(DocumentDto dto);

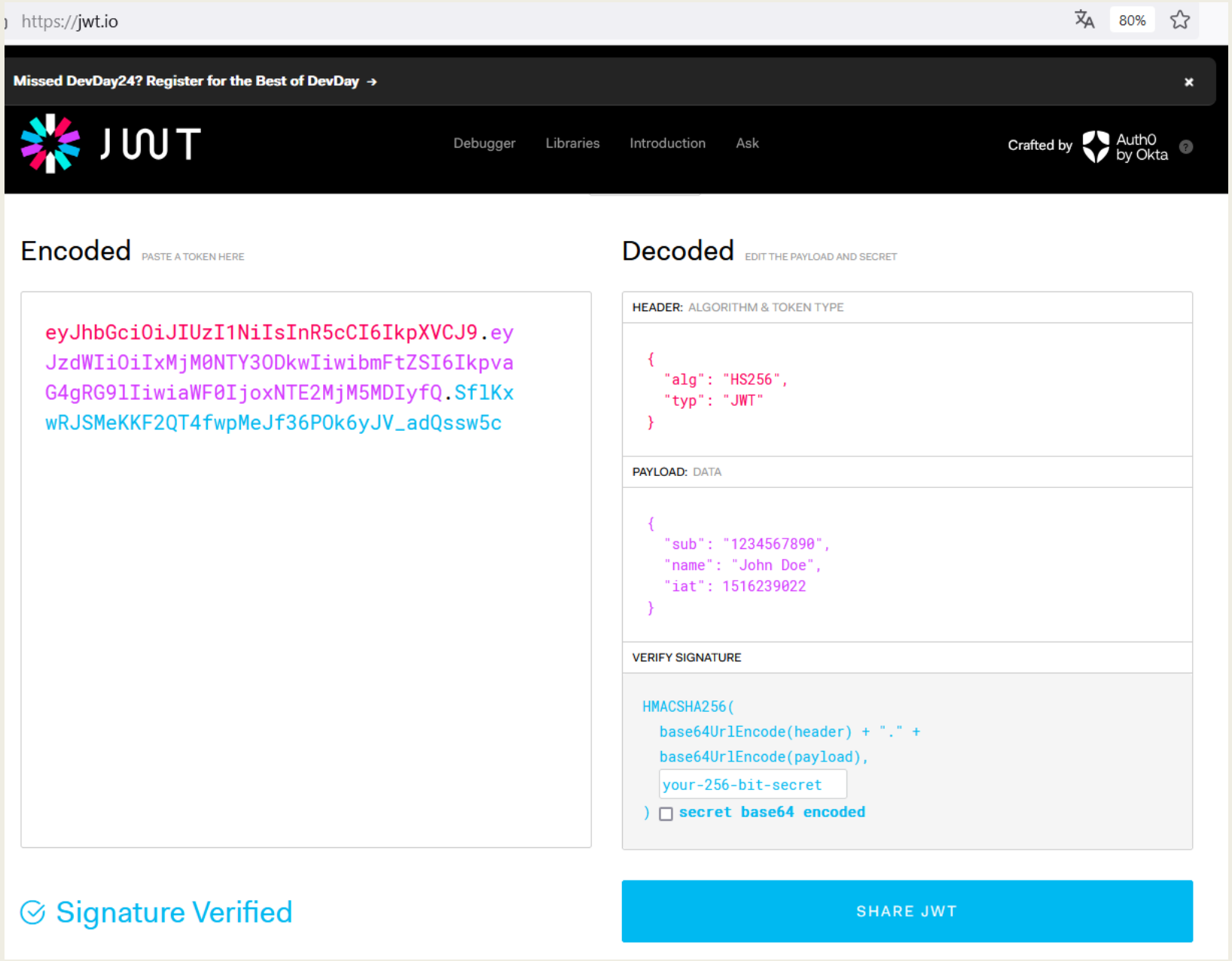

   @PreAuthorize(“@checkerBean.checkPermissionDocumentDelete (#id)”)
    void deleteDocument(UUID id);

}
```

Выводы


UserDetails Service
InMemory UserDetails Manager
JDBC UserDetails Manager
LDAP UserDetails Manager
...
load User By Username()
User Details
Providers
Dao Authentication Provider
OAuth2Login Authentication Provider
LDAP Authentication Provider
...
Password Encoder
BCrypt
Argon2
Authentication
Authentication Manager
Provider Manager
Authentication Token
Username Password Authentication Token
SecurityContext HolderFilter
UsernamePassword AuthenticationFilter
Exception TranslationFilter

Выводы


https://jwt.io
80%
Missed DevDay24? Register for the Best of DevDay →
JWT
Debugger
Libraries
Introduction
Ask
Crafted by
Auth0 by Okta
Encoded
PASTE A TOKEN HERE
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
Decoded
EDIT THE PAYLOAD AND SECRET
HEADER: ALGORITHM & TOKEN TYPE
{
"alg": "HS256",
"typ": "JWT"
}
PAYLOAD: DATA
{
"sub": "1234567890",
"name": "John Doe",
"iat": 1516239022
}
VERIFY SIGNATURE
HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),
your-256-bit-secret
) secret base64 encoded
Signature Verified
SHARE JWT